

Worcester Polytechnic Institute

Robotics Engineering (RBE 4701)

Project Title: Bomberman/Gridworld Safe-Route Planner

Team 3 – Bryce Lukacs, Benni Valentine, Amir Sefiyaoui

Professor: Jing Xiao

Submission Date: September 16, 2025

Repository: <https://github.com/MDevVal/RBE470x-projects/tree/bryce/v2-v5-fixes> (project folder: <https://github.com/MDevVal/RBE470x-projects/tree/bryce/v2-v5-fixes/team03>)

Abstract

We worked on a pathfinding agent for the Bomberman gridworld. The project focused on getting from start to goal without dying to bombs or blast zones as well as avoiding monsters. We created an A* agent to avoid danger by sometimes waiting a few steps or using bombs to kill monsters. We tested our agent in the provided Python game setup and had to fix and tune our minimax and A* to 1 be able to plant bombs and avoid monsters, but 2, make it to goal in a quick fashion. We ended up performing better on maps with hazards like Variant 3 and 5, and had the most difficulty with variant 4. Overall, the planner worked how we expected, and we were successfully able to beat the game.

1. Structure of the Approach

1.1 Finite-State Behavior (lightweight)

Instead of one fixed policy, **Team3Char** changes what it's doing based on the situation it's in:

- **Explore / PathFollow:** Uses A* to head for the exit and we gave it extra cost for risky tiles.
 - **AvoidHazard:** If the next A* step looks sketchy (closer to a monster), it will take a safer step that still moves towards the goal.
 - **EscapeAfterBomb:** When we drop a bomb, follow a planned escape route until the blast is over.
 - **AttackEnemy:** Only place a bomb if an enemy is in range *and* we already have a safe and timed escape.
 - **WaitForBlastClear:** If every option is bad, we'll briefly wait rather than walk into danger.
-

2. Implementation Details

2.1 Enemy Avoidance

Our A* risk model includes predicted enemy movement using minimax. If an enemy was likely to move into a tile, that tile's cost was raised, and the agent would avoid it. This prediction was only a few steps ahead but was enough to improve survival.

2.2 Bomb Placement Logic

The agent would place a bomb if an enemy was within range **and** an escape route was available. After planting a bomb, it switched to **avoid** state and replanned to avoid its own blast.

2.3 Wait Behavior

When surrounded by hazards with no immediate path to safety, the agent could insert “wait” actions. These were useful for timing bombs or letting enemies move out of the way.

2.4 Risk Weight Tuning and Fixes

- Risk too low: agent ran into blast zones
 - Risk too high: agent froze or over-waited or paced back and forth at top corner
-

3. Experimental Evaluation

3.1 Setup

- All runs were done using the provided framework.
- Platform: Windows 11, Open BSD, Python 3.12, Pygame 2.6.1
- Each variant was tested 10 times, using randomized seeds where applicable.

3.2 Results Table

Variant	Successes (out of 10)	Notes
1	10/10	Open map; A* + soft costs reach exit quickly.
2	10/10	Prior corner pacing fixed by evasion scoring; avoids stupid monster pretty reliably.
3	10/10	Hazards handled well; soft costs and diagonal A* give clean routes.
4	0/10	Aggressive corners us in tight corridors; short horizon threat modeling is not sufficient.
5	10/10	Mixed hazards; agent's avoidance is robust and reaches exit.

3.3 Key Observations

- Variants 3 and 5 showed big gains from A* and the state machine.
 - BFS failed often in dynamic maps due to lack of hazard modeling.
 - Variant 5 exposed edge cases in our blast prediction logic.
-

Appendix A – Sample Logs

TIME LEFT: 4973

+-----+

S	
	A
	C
	#

+-----+

SCORES

me -4973

EVENTS

TIME LEFT: 4972

+-----+

S	
	A

| |

| |

| #|

+-----+

SCORES

me 4971

EVENTS

me found the exit

Process finished with exit code 0